Decoupling Solutions on AWS

This week's customer had issues with latency for their end users because they were using a synchronous model for their web backend. Morgan suggested that they migrate to an asynchronous model, where the order is first stored, and then processed shortly after. With this model, end user can receive a response more quickly from the backend. When the backend system receives an order, it can immediately respond and then process the request asynchronously.

A loosely coupled architecture minimizes the bottlenecks that are caused by synchronous communication, latency, and I/O operations. Amazon Simple Queue Service (Amazon SQS) and AWS Lambda are often used to implement asynchronous communication between different services.You should consider using this pattern if you have the following requirements:

- You want to create loosely coupled architecture.
- All operations don't need to be completed in a single transaction, and some operations can be asynchronous.
- The downstream system can't handle the incoming transactions per second (TPS) rate. The messages can be written to the queue and processed based on the availability of resources.

A disadvantage of this pattern is that the actions of business transaction are synchronous. Even though the calling system receives a response, some part of the transaction might still continue to be processed by downstream systems.

For more information, see Patterns for integrating microservices.

Building storage-first applications

Building a storage-first application can help an application be more reliable when working with events. You can read more about this topic in the blog post
Building storage-first serverless applications with HTTP APIs service integrations.

Amazon SQS

Amazon SQS is a fully managed, message queuing service that you can use to decouple and scale microservices, distributed systems, and serverless applications. Amazon SQS reduces the complexity and overhead associated with managing and operating message-oriented middleware, which means that developers can focus on differentiating work. By using Amazon SQS, you can send, store, and receive messages between software components at virtually any volume, without losing messages or requiring other services to be available.

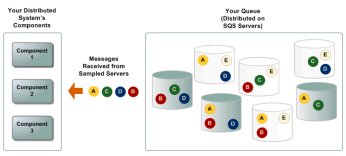
When you create or edit a queue, you can configure the following parameters:

- Visibility timeout: The length of time that a message received from a queue (by one consumer) won't be visible to the other message consumers.
- Message retention period: The amount of time that Amazon SQS retains messages that remain in the queue. By default, the queue retains messages for 4 days. You can configure a queue to retain messages for up to 14 days.
- Delivery delay: The amount of time that Amazon SQS will delay before it delivers a message that is added to the queue. For more information, see Amazon SQS delay queues.
- Maximum message size: The maximum message size for the queue.
- Receive message wait time: The maximum amount of time that Amazon SQS waits for messages to become available after the queue gets a receive request.
- Enable content-based deduplication: Amazon SQS can automatically create deduplication IDs based on the body of the message.
- Enable high throughput FIFO: This feature enables high throughput for messages in the queue. Choosing this option changes the related options (deduplication scope and FIFO throughput limit) to the required settings for enabling high throughput for FIFO queues.
- Redrive allow policy: This policy defines which source queues can use this queue as the dead-letter queue.

Short polling compared to long polling

When you consume messages from a queue by using short polling, Amazon SQS samples a subset of its servers (based on a weighted random distribution) and returns messages from only those servers. Thus, a particular ReceiveMessage request might not return all of your messages. However, if you have fewer than 1,000 messages in your queue, a subsequent request will return your messages. If you keep consuming from your queues, Amazon SQS samples all of its servers, and you receive all of your messages.

The following diagram shows the short-polling behavior of messages returned from a standard queue after one of your system components makes a receive request. Amazon SQS samples several of its servers (in gray) and returns messages A, C, D, and B from these servers. Message E isn't returned for this request, but it's returned for a subsequent request.



When the wait time for the ReceiveMessage API action is greater than 0, *long polling* is in effect. The maximum long polling wait time is 20 seconds. Long polling helps reduce the cost of using Amazon SQS by reducing the number of empty responses (when there are no messages available for a ReceiveMessage request) and false empty responses (when messages are available, but aren't included in a response). For information about enabling long polling for a new or existing queue using the Amazon SQS console, see Configuring queue parameters (console). For best practices, see Setting up long polling.

Long polling offers the following benefits:

- Reduces empty responses by letting Amazon SQS wait until a message is available in a queue before it sends a response. Unless the connection times out, the response to the ReceiveMessage request contains at least one of the available messages, up to the maximum number of messages specified in the ReceiveMessage action. In rare cases, you might receive empty responses even when a queue still contains messages, especially if you specify a low value for the ReceiveMessageWaitTimeSeconds parameter.
- Reduces false empty responses by querying all—instead of a subset of—Amazon SQS servers.
- Returns messages as soon as they become available.

Resources

- For more information about Amazon SQS, see What is Amazon Simple Queue Service? or Amazon SQS FAQs.
- For a tutorial about setting up Amazon SQS to invoke Lambda functions, see Using AWS Lambda with Amazon SQS.

Mark as completed

Like Dislike Report an issue

Go to next item

coach

Hi, Venkatesh!
How can I help?

Explain this topic in simple terms | Give me a summary | Give me practice questions | Give me real-life examples

Ask me anything